Package 'levysim'

October 22, 2026

Title Simulation of Degradation Processes

Version 0.0.0.9000

Description Exact simulation of one or several degradation trajectories of gamma, Wiener, inverse Gaussian, variance-gamma, Meixner and inverse gamma processes, homogeneous or not, at possibly irregular inspection times; plotting of the trajectories and extraction of the simulated data.

License GPL-3

URL https://github.com/rytracetito/levy_packages/tree/main/levysim, https://github.com/rytracetito/levy_packages

BugReports https://github.com/rytracetito/levy_packages/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports ggplot2, Runuran, stats, utils

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Author Amedée Rytrace Tito [aut, cre], Christian Paroissin [aut]

Maintainer Amedée Rytrace Tito <rytracetito5@gmail.com>

Contents

levysim-package 2
inspection_times 4
plot_plan 6
plot_traj 7
sim_gamma 8
sim_ig 10
sim_igam 12
sim_meixner 15
sim_vg 17
sim_wiener 20
traj_df 22

Index **24**

`levysim-package` *levysim: simulation of degradation processes*

Description

`levysim` simulates exactly one or several degradation trajectories of six stochastic processes, homogeneous (Lévy processes) or not, at inspection times that can be irregular and differ from one trajectory to another. The simulated trajectories can be plotted and extracted as a data frame.

Functions of the package

Simulation (click on a function name for its help page):

Function	Process	Parameters	Functions of time
`sim_gamma()`	gamma	shape `a`, rate `b`	`A`
`sim_wiener()`	Wiener with drift	`mu`, `sigma`	`Lambda`, `tau`
`sim_ig()`	inverse Gaussian	`mu`, `eta`	`Lambda`
`sim_vg()`	variance-gamma	`theta`, `sigma`, `nu`	`Lambda`
`sim_meixner()`	Meixner	`a`, `b`, `d`, `m`	`Lambda`
`sim_igam()`	inverse gamma	`theta`, `alpha`, `beta`	`eta_inv`

The functions of time are the identity by default: the processes are then homogeneous, that is Lévy processes (except the inverse gamma process, which is never a Lévy process).

Simulation methods:

- gamma: `stats::rgamma()`; Wiener: `stats::rnorm()`;
- inverse Gaussian: method of Michael, Schucany and Haas;
- variance-gamma: subordination, or difference of two gamma processes (argument `method`);
- Meixner: numerical inversion of the distribution function (`Runuran`);
- inverse gamma: exact passage algorithm.

Results and inspection schedules:

- `plot_traj()`: plot of the simulated trajectories (`ggplot2`);
- `plot_plan()`: plot of the inspection times, one row per trajectory (`ggplot2`);
- `traj_df()`: simulated data as a data frame with columns `id`, `t`, `X`, and optionally the time steps `dt` and increments `dX`;
- `inspection_times()`: inspection times with a regular, increasing, decreasing or alternating number of inspections over time.

Getting started

A simulation always follows the same three steps.

1. **Choose the inspection times** `t`. Three forms are accepted by every `sim_` function:
   - a vector, for instance `seq(0, 10, by = 0.5)`: all the trajectories share these times, and their number is given by `n_traj`;
   - a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`;

- the result of `inspection_times()`, for instance with the pattern `"increasing"` for trajectories inspected more and more often.

The times of each trajectory start at 0 and are strictly increasing.

2. **Simulate** with the function of the chosen process and its parameters, for instance `sim_gamma()` for a gamma process. For a non-homogeneous process, give the function of time, for instance `A = function(t) t^1.5`.
3. **Store and use the result** `x`. Its form depends on the times:
   - same times for all the trajectories: a matrix with the column `t` and one column per trajectory, `X1`, ..., `Xn`; the k-th trajectory is `x[, "Xk"]`;
   - different times: a list with one data frame (`t`, `X`) per trajectory; the k-th trajectory is `x[[k]]`.

   In both cases, `x` prints the model, the parameters and the first simulated values, `traj_df(x)` gives all the data in one data frame (`id`, `t`, `X`, and `dt`, `dX` with `increments = TRUE`) and `plot_traj(x)` plots the trajectories.

Use `set.seed()` before a simulation to make it reproducible.

Exact simulation

For the first five processes, which have independent increments, the joint distribution at the inspection times is determined by the distributions of the increments: the simulation draws independent increments with the prescribed distributions and cumulates them, without any time discretization. The inverse gamma process, whose increments are dependent, uses a specific exact algorithm.

Author(s)

Maintainer: Amedée Rytrace Tito <rytracetito5@gmail.com>

Authors:

- Christian Paroissin <christian.paroissin@univ-pau.fr>

References

Gamma process: van Noortwijk, J. M. (2009). A survey of the application of gamma processes in maintenance. *Reliability Engineering & System Safety*, 94(1), 2-21. doi:10.1016/j.ress.2007.03.019

Wiener process: Whitmore, G. A. (1995). Estimating degradation by a Wiener diffusion process subject to measurement error. *Lifetime Data Analysis*, 1, 307-319.

Inverse Gaussian process: Wang, X. and Xu, D. (2010). An inverse Gaussian process model for degradation data. *Technometrics*, 52(2), 188-197. doi:10.1198/TECH.2009.08197

Variance-gamma process: Madan, D. B., Carr, P. P. and Chang, E. C. (1998). The variance gamma process and option pricing. *European Finance Review*, 2(1), 79-105. doi:10.1023/A:1009703431535

Meixner process: Schoutens, W. and Teugels, J. L. (1998). Lévy processes, polynomials and martingales. *Communications in Statistics. Stochastic Models*, 14(1-2), 335-349.

Inverse gamma process: Guida, M. and Pulcini, G. (2013). The inverse Gamma process: a family of continuous stochastic models for describing state-dependent deterioration phenomena. *Reliability Engineering & System Safety*, 120, 72-79. doi:10.1016/j.ress.2013.03.013

The references of each method are given in the help page of each function.

See Also

Useful links:

- https://github.com/rytracetito/levy_packages/tree/main/levysim
- https://github.com/rytracetito/levy_packages
- Report bugs at https://github.com/rytracetito/levy_packages/issues

Examples

```
# 1. Inspection times: 10 trajectories, inspected more and more often
plan <- inspection_times("increasing", n = 20, n_traj = 10, tmax = 100)
plot_plan(plan)

# 2. Simulation of a homogeneous gamma process
set.seed(1)
x <- sim_gamma(plan, a = 2, b = 3)

# 3. Use of the result: the same times for all the trajectories give a
#    matrix (column t, then X1, ..., X10)
x
x[, "X1"]
head(traj_df(x, increments = TRUE))
plot_traj(x)
```

---

inspection_times *Inspection times with a chosen pattern*

Description

Generates inspection times on $[0, t_{max}]$ for one or several trajectories, with a number of inspections per unit of time that is constant, increasing, decreasing or alternating.

Usage

```
inspection_times(
  pattern = "regular",
  n = 10,
  n_traj = 1,
  tmax = 100,
  digits = NULL,
  a = 0.5
)
```

Arguments

pattern	Name of the pattern (see Details) or a strictly increasing function on $[0, 10]$.
n	Number of inspections after time 0, for all trajectories or one per trajectory.
n_traj	Number of trajectories.
tmax	Time of the last inspection, for all trajectories or one per trajectory.
digits	Number of decimal places of the times, NULL for no rounding. An error is raised if rounding makes two times equal.

a — Amplitude of the sine in the four alternating patterns: it sets the contrast between frequent and rare inspections, from 0 (regular inspections) to 1 excluded; default 0.5. Not used by the other patterns.

Details

The times are the values of an increasing function f on a regular grid $x_i = i\,x_{max}/n$, $i = 0, \ldots, n$, with $x_{max} = 10$, rescaled to $[0, t_{max}]$:

$$t_i = t_{max}\frac{f(x_i) - f(0)}{f(x_{max}) - f(0)}.$$

The gap between two inspections is about $f'(x)\,x_{max}/n$: the inspections are frequent where f is flat and rare where f is steep. The patterns are

- `"regular"`: $f(x) = x$, constant gaps;
- `"increasing"`: $f(x) = \ln(x + 1)$, more and more inspections over time;
- `"decreasing"`: $f(x) = -\ln(x_{max} + 1 - x)$, fewer and fewer inspections, mirror image of `"increasing"`;
- `"inc-dec"`: $f(x) = x + a\,\frac{x_{max}}{2\pi}\sin(2\pi x/x_{max})$, the number of inspections increases then decreases: frequent in the middle, rare at the start and at the end;
- `"dec-inc"`: same with a minus sign, the number of inspections decreases then increases: frequent at the start and at the end;
- `"inc-dec-inc"`: $f(x) = x + a\,\frac{x_{max}}{3\pi}\sin(3\pi x/x_{max})$, the number of inspections increases, decreases, then increases;
- `"dec-inc-dec"`: same with a minus sign.

For the four alternating patterns, $f'(x) = 1 \pm a\cos(k\pi x/x_{max})$: the gaps vary between $1 - a$ and $1 + a$ times the mean gap, so that the largest gap is $(1 + a)/(1 - a)$ times the smallest one, where a is the amplitude `a`. With the default $a = 0.5$, this ratio is 3 and the gaps change progressively. With $a = 0.8$, the ratio is 9: the frequent inspections gather in a short time, which looks like a break in the plan, especially with three phases and few inspections. $a = 0$ gives regular inspections. Three phases need one and a half oscillations of the sine ($k = 3$), two phases one oscillation ($k = 2$). With three phases and few inspections (fewer than 20), choose `a` between 0.3 and 0.5: even with 0.5, ten inspections give a jump of about 1.6 between two consecutive gaps.

Any other strictly increasing function on $[0, 10]$ can be given as `pattern`.

Value

A data frame with columns `id` (trajectory) and `time`, time 0 included.

See Also

`plot_plan()` to plot the inspection times

Examples

```
# Regular inspections: 10 inspections after time 0, up to 100
inspection_times("regular", n = 10, tmax = 100)

# More and more frequent inspections, times rounded to integers
inspection_times("increasing", n = 10, digits = 0)
```

```
# Fewer and fewer inspections
inspection_times("decreasing", n = 10, digits = 0)

# Frequent in the middle of the period, rare at the start and at the end
inspection_times("inc-dec", n = 20, digits = 1)

# Frequent at the start and at the end, rare in the middle
inspection_times("dec-inc", n = 20, digits = 1)

# Three phases: frequent, rare, frequent, rare
inspection_times("dec-inc-dec", n = 20, digits = 1)

# Stronger contrast between frequent and rare inspections
inspection_times("dec-inc-dec", n = 20, digits = 1, a = 0.8)

# Gaps between inspections: from about 2.5 to 7.5 with the default amplitude
range(diff(inspection_times("dec-inc-dec", n = 20)$time))

# Three trajectories with 10, 15 and 20 inspections, up to 50, 100 and 150
plan <- inspection_times("increasing", n = c(10, 15, 20), n_traj = 3,
                         tmax = c(50, 100, 150), digits = 1)
plan

# Times of the second trajectory only
plan$time[plan$id == 2]

# Any strictly increasing function on [0, 10]: with sqrt, the inspections
# are rare at the start and more and more frequent
inspection_times(sqrt, n = 10, digits = 0)

# Mixture of the increasing and decreasing patterns: frequent inspections
# around a point that moves to the right when w increases (0 < w < 1)
w <- 0.8
inspection_times(function(x) w * log(x + 1) - (1 - w) * log(11 - x), n = 20, digits = 1)

# Plot the inspection times, one row per trajectory
plan <- inspection_times("dec-inc", n = c(10, 20, 30), n_traj = 3, tmax = 100)
plot_plan(plan)
```

`plot_plan` *Plot inspection times*

Description

Draws the inspection times of one or several trajectories with `ggplot2`: one point per inspection, and one row per trajectory when there are several. It shows at a glance where the inspections are frequent and where they are rare.

Usage

```
plot_plan(t, title = "Inspection times")
```

Arguments

t	Inspection times, in one of the three forms accepted by the sim_ functions: • a numeric vector, for one trajectory, for instance seq(0, 10, by = 0.5); • a list of numeric vectors, one per trajectory, for instance list(c(0, 1, 3), c(0, 2, 4, 6)); • the result of inspection_times(), one plan per trajectory. The times of each trajectory start at 0 and are strictly increasing.
title	Title of the plot, a character string; by default "Inspection times".

Value

A ggplot object. It is displayed when printed, and it can be modified with the usual ggplot2 functions, added with +: for instance + ggplot2::labs(x = "Time (hours)") to change the axis label.

See Also

inspection_times(), plot_traj()

Examples

```
# One trajectory inspected every 0.5 time unit
plot_plan(seq(0, 10, by = 0.5))

# Inspections more and more frequent over time, with a custom title
t <- inspection_times("increasing", n = 30, tmax = 100)
plot_plan(t, "Increasing number of inspections")

# Five trajectories, each with its own times: one row per trajectory
plan <- inspection_times("dec-inc", n = c(10, 15, 20, 25, 30), n_traj = 5, tmax = 100)
plot_plan(plan)

# Different inspection times given as a list
plot_plan(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)))

# Change the axis label
plot_plan(t) + ggplot2::labs(x = "Time (hours)")
```

---

plot_traj *Plot simulated trajectories*

Description

Draws the trajectories simulated by a sim_ function with ggplot2: one grey line per trajectory, with a point at each inspection.

Usage

```
plot_traj(x, title = NULL)
```

Arguments

x	Result of a simulation function of the package, for instance sim_gamma().
title	Title of the plot, a character string; by default the name of the process, for instance "gamma process".

Value

A ggplot object. It is displayed when printed, and it can be modified with the usual ggplot2 functions, added with +: for instance + ggplot2::geom_hline(yintercept = 10) to draw a failure threshold, or + ggplot2::labs(x = "Time (hours)") to change an axis label.

See Also

traj_df()

Examples

```
set.seed(1)
x <- sim_wiener(seq(0, 10, by = 0.5), mu = 0.5, sigma = 1.5, n_traj = 5)
plot_traj(x)

# Custom title
plot_traj(x, "Five trajectories of a Wiener process")

# Add a failure threshold and change the axis labels
plot_traj(x) +
  ggplot2::geom_hline(yintercept = 5, linetype = "dashed", colour = "red") +
  ggplot2::labs(x = "Time (hours)", y = "Degradation (%)")
```

---

sim_gamma *Simulation of the gamma process*

Description

Simulates exactly one or several trajectories of a gamma process, homogeneous or not, at given inspection times.

Usage

```
sim_gamma(t, a, b, A = function(t) t, n_traj = 1)
```

Arguments

t	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector, for instance seq(0, 10, by = 0.5): all the trajectories share these times, and their number is n_traj; • a list of vectors, one per trajectory, for instance list(c(0, 1, 3), c(0, 2, 4, 6)): each trajectory has its own times (n_traj is then ignored); • the data frame returned by inspection_times(), which gives one plan per trajectory.

a	Shape parameter, $a > 0$.
b	Rate parameter, $b > 0$ (the scale is $1/b$).
A	Shape function of time: an R function of t, non-decreasing with $A(0) = 0$, for instance function(t) t^1.5. The default function(t) t gives the homogeneous process.
n_traj	Number of trajectories to simulate when t is a vector (default 1).

Value

An object of class levysim, whose form depends on the inspection times:

- **same times for all the trajectories** (t a vector, or a plan of inspection_times() with the same times for all the trajectories): a matrix with columns t (inspection times), X1, ..., Xn (simulated degradation levels of the trajectories 1 to n, equal to 0 at t = 0). The k-th trajectory is x[, k + 1] or x[, "Xk"];
- **different times** (t a list, or a plan of inspection_times() with different times): a list with one data frame per trajectory, with columns t and X. The k-th trajectory is x[[k]].

Printing the object shows the model, the parameters and the first rows. Use traj_df() to gather the data in a single data frame (with the increments if needed) and plot_traj() to plot the trajectories.

Model

Let A be a non-decreasing function with $A(0) = 0$, $a > 0$ and $b > 0$. The gamma process satisfies $X(0) = 0$, has independent increments, and for $0 \leq s < t$,

$$X(t) - X(s) \sim \Gamma\big(a(A(t) - A(s)),\ b\big),$$

where $\Gamma(k, b)$ is the gamma distribution with shape k and **rate** b, of density $b^k x^{k-1} e^{-bx}/\Gamma(k)$. Then $E[X(t)] = aA(t)/b$ and $Var(X(t)) = aA(t)/b^2$. The process is increasing and purely made of jumps. With $A(t) = t$ (default), it is the homogeneous gamma process, a Lévy process. A classical non-homogeneous choice is $A(t) = t^\beta$.

Exact simulation

1. $\Delta A_i = A(t_i) - A(t_{i-1})$, $i = 1, \ldots, n$.
2. Draw independently $\Delta X_i \sim \Gamma(a\Delta A_i, b)$.
3. $X(t_0) = 0$ and $X(t_i) = \sum_{j \leq i} \Delta X_j$.

References

van Noortwijk, J. M. (2009). A survey of the application of gamma processes in maintenance. *Reliability Engineering & System Safety*, 94(1), 2-21. doi:10.1016/j.ress.2007.03.019

See Also

plot_traj(), traj_df(), inspection_times(), plot_plan()

Examples

```
# One trajectory inspected every 0.5 time unit up to 10
t <- seq(0, 10, by = 0.5)
set.seed(1)
x <- sim_gamma(t, a = 2, b = 3)
x
plot_traj(x)

# Ten trajectories with the same inspection times
x <- sim_gamma(t, a = 2, b = 3, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])
x[, "X2"]

# Irregular inspections, more and more frequent over time
plan <- inspection_times("increasing", n = 20, n_traj = 5, tmax = 10)
x <- sim_gamma(plan, a = 2, b = 3)
plot_traj(x)

# Different inspection times for each trajectory: a list of data frames
x <- sim_gamma(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), a = 2, b = 3)
x[[2]]

# Non-homogeneous process with A(t) = t^1.5
x <- sim_gamma(t, a = 2, b = 3, A = function(t) t^1.5, n_traj = 10)
plot_traj(x, "Non-homogeneous gamma process")

# All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))
```

`sim_ig` *Simulation of the inverse Gaussian process*

Description

Simulates exactly one or several trajectories of an inverse Gaussian (IG) process, homogeneous or not, at given inspection times.

Usage

```
sim_ig(t, mu, eta, Lambda = function(t) t, n_traj = 1)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector, for instance `seq(0, 10, by = 0.5)`: all the trajectories share these times, and their number is `n_traj`;

- a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` is then ignored);
- the data frame returned by `inspection_times()`, which gives one plan per trajectory.

`mu` Mean degradation per unit of Λ, $\mu > 0$.

`eta` Shape parameter, $\eta > 0$. The variance of $X(t)$ is $\mu^3\Lambda(t)/\eta$: the larger η, the less variable the increments.

`Lambda` Function of time: an R function of `t`, continuous and strictly increasing with $\Lambda(0) = 0$, for instance `function(t) t^1.5`. The default `function(t) t` gives the homogeneous process.

`n_traj` Number of trajectories to simulate when `t` is a vector (default 1).

Value

An object of class `levysim`, whose form depends on the inspection times:

- **same times for all the trajectories** (`t` a vector, or a plan of `inspection_times()` with the same times for all the trajectories): a matrix with columns `t` (inspection times), `X1`, ..., `Xn` (simulated degradation levels of the trajectories 1 to n, equal to 0 at `t = 0`). The k-th trajectory is `x[, k + 1]` or `x[, "Xk"]`;
- **different times** (`t` a list, or a plan of `inspection_times()` with different times): a list with one data frame per trajectory, with columns `t` and `X`. The k-th trajectory is `x[[k]]`.

Printing the object shows the model, the parameters and the first rows. Use `traj_df()` to gather the data in a single data frame (with the increments if needed) and `plot_traj()` to plot the trajectories.

Model

$IG(m, \lambda)$ denotes the inverse Gaussian distribution with mean m and shape λ, of density

$$\sqrt{\frac{\lambda}{2\pi x^3}}\exp\Big(-\frac{\lambda(x-m)^2}{2m^2x}\Big), \qquad x > 0.$$

Let Λ be continuous, strictly increasing, with $\Lambda(0) = 0$, $\mu > 0$ and $\eta > 0$. The IG process has $X(0) = 0$, independent increments and

$$X(t) - X(s) \sim IG(\mu\Delta\Lambda, \eta\Delta\Lambda^2).$$

Hence $E[X(t)] = \mu\Lambda(t)$ and $Var(X(t)) = \mu^3\Lambda(t)/\eta$. The process is increasing and purely made of jumps. With $\Lambda(t) = t$ (default), it is the homogeneous IG process, a Lévy process, with the parametrization of Wang and Xu (2010).

Exact simulation

Each increment $IG(m, \lambda)$, with $m = \mu\Delta\Lambda_i$ and $\lambda = \eta\Delta\Lambda_i^2$, is drawn by the method of Michael, Schucany and Haas (1976), based on $\lambda(X-m)^2/(m^2X) \sim \chi^2_1$:

1. $\nu = Z^2$, $Z \sim N(0,1)$;
2. $x_1 = m + m^2\nu/(2\lambda) - m\sqrt{4m\lambda\nu + m^2\nu^2}/(2\lambda)$;
3. return x_1 with probability $m/(m + x_1)$, otherwise m^2/x_1.

The increments are independent; the trajectory is their cumulative sum.

References

Michael, J. R., Schucany, W. R. and Haas, R. W. (1976). Generating random variates using transformations with multiple roots. *The American Statistician*, 30(2), 88-90. doi:10.1080/00031305.1976.10479147

Wang, X. and Xu, D. (2010). An inverse Gaussian process model for degradation data. *Technometrics*, 52(2), 188-197. doi:10.1198/TECH.2009.08197

See Also

`plot_traj()`, `traj_df()`, `inspection_times()`, `plot_plan()`

Examples

```
# One trajectory inspected every 0.5 time unit up to 10
t <- seq(0, 10, by = 0.5)
set.seed(1)
x <- sim_ig(t, mu = 0.5, eta = 2)
x
plot_traj(x)

# Ten trajectories with the same inspection times
x <- sim_ig(t, mu = 0.5, eta = 2, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])
x[, "X2"]

# Irregular inspections, less and less frequent over time
plan <- inspection_times("decreasing", n = 20, n_traj = 5, tmax = 10)
x <- sim_ig(plan, mu = 0.5, eta = 2)
plot_traj(x)

# Different inspection times for each trajectory: a list of data frames
x <- sim_ig(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), mu = 0.5, eta = 2)
x[[2]]

# Non-homogeneous process with Lambda(t) = t^1.5
x <- sim_ig(t, mu = 0.5, eta = 2, Lambda = function(t) t^1.5, n_traj = 10)
plot_traj(x, "Non-homogeneous inverse Gaussian process")

# All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))
```

`sim_igam` *Simulation of the inverse gamma process*

Description

Simulates exactly one or several trajectories of the inverse gamma process at given inspection times.

Usage

```
sim_igam(t, theta, alpha = 1, beta = 1, eta_inv = NULL, n_traj = 1)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector, for instance `seq(0, 10, by = 0.5)`: all the trajectories share these times, and their number is `n_traj`; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` is then ignored); • the data frame returned by `inspection_times()`, which gives one plan per trajectory.
`theta`	Scale of the gamma process in level, $\theta > 0$: the larger θ, the slower the degradation.
`alpha, beta`	Parameters of the default shape function $\eta(w) = (w/\alpha)^\beta$, $\alpha > 0$, $\beta > 0$ (default 1 for both).
`eta_inv`	Optional: an R function giving the inverse $\eta^{-1}(u)$ of another shape function, for instance `function(u) log(1 + u)` for $\eta(w) = e^w - 1$. It then replaces $(w/\alpha)^\beta$, and `alpha` and `beta` are ignored.
`n_traj`	Number of trajectories to simulate when `t` is a vector (default 1).

Value

An object of class `levysim`, whose form depends on the inspection times:

- **same times for all the trajectories** (`t` a vector, or a plan of `inspection_times()` with the same times for all the trajectories): a matrix with columns `t` (inspection times), `X1`, ..., `Xn` (simulated degradation levels of the trajectories 1 to n, equal to 0 at `t = 0`). The k-th trajectory is `x[, k + 1]` or `x[, "Xk"]`;
- **different times** (`t` a list, or a plan of `inspection_times()` with different times): a list with one data frame per trajectory, with columns `t` and `X`. The k-th trajectory is `x[[k]]`.

Printing the object shows the model, the parameters and the first rows. Use `traj_df()` to gather the data in a single data frame (with the increments if needed) and `plot_traj()` to plot the trajectories.

Model

Let η be continuous, strictly increasing, with $\eta(0) = 0$, and $\theta > 0$. Let $T(w)$ be a gamma process indexed by the degradation level w, with $T(w_2) - T(w_1) \sim \Gamma(\eta(w_2) - \eta(w_1))$ with scale θ: $T(w)$ is the time needed to reach the level w. The inverse gamma process is

$$W(t) = \inf\{w \geq 0 : T(w) > t\}.$$

Writing $T(w) = \theta G(\eta(w))$ with G a standard gamma process gives $W(t) = \eta^{-1}(L(t/\theta))$, with $L(z) = \inf\{u : G(u) > z\}$, and the marginal distribution $P(W(t) \leq w) = Q(\eta(w), t/\theta)$, where Q is the regularized upper incomplete gamma function. W is continuous, increasing and constant on long plateaus: two inspections can give the same value with positive probability. It is neither a Lévy process nor a Markov process. By default $\eta(w) = (w/\alpha)^\beta$.

Exact simulation

The increments are not independent. In reduced scale $z_j = t_j/\theta$, the algorithm follows the position of G just after the last jump that crossed a threshold; after such a jump, G starts afresh (strong Markov property). For a standard gamma process starting at 0, the passage level $s = L(r)$ above r satisfies $P(L(r) \leq s) = Q(s, r)$; given s, the position before the jump $y \in (0, r)$ has a density proportional to $g_s(y)E_1(r - y)$ (g_s the density of $\Gamma(s, 1)$, E_1 the exponential integral); given y, the jump $J > r - y$ satisfies $P(J > j) = E_1(j)/E_1(r - y)$.

1. Start with level $a = 0$ and position $pos = 0$.
2. For each inspection j: if $pos > z_j$, the previous jump covers z_j and $a_j = a_{j-1}$ (equal observations); otherwise, with $r = z_j - pos$, draw s by inversion of $Q(s, r)$, then y, then J, and set $a_j = a_{j-1} + s$, $pos = pos + y + J$.
3. Return $W(t_j) = \eta^{-1}(a_j)$.

The algorithm is exact up to numerical errors (quadrature on 401 nodes, bisection, root finding).

References

Guida, M. and Pulcini, G. (2013). The inverse Gamma process: a family of continuous stochastic models for describing state-dependent deterioration phenomena. *Reliability Engineering & System Safety*, 120, 72-79. doi:10.1016/j.ress.2013.03.013

See Also

`plot_traj()`, `traj_df()`, `inspection_times()`, `plot_plan()`

Examples

```
# One trajectory inspected every time unit up to 10
t <- seq(0, 10, by = 1)
set.seed(1)
x <- sim_igam(t, theta = 1, alpha = 1, beta = 1.5)
x
plot_traj(x)

# Five trajectories with the same inspection times: note the plateaus
x <- sim_igam(t, theta = 1, alpha = 1, beta = 1.5, n_traj = 5)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])
x[, "X2"]

# Irregular inspections, more and more frequent over time
plan <- inspection_times("increasing", n = 10, n_traj = 3, tmax = 10)
x <- sim_igam(plan, theta = 1, alpha = 1, beta = 1.5)
plot_traj(x)

# Different inspection times for each trajectory: a list of data frames
x <- sim_igam(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), theta = 1, alpha = 1, beta = 1.5)
x[[2]]

# Another shape function: eta(w) = exp(w) - 1, given by its inverse
x <- sim_igam(t, theta = 1, eta_inv = function(u) log(1 + u), n_traj = 5)
```

```
plot_traj(x, "Inverse gamma process with eta(w) = exp(w) - 1")

# All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))
```

`sim_meixner` *Simulation of the Meixner process*

Description

Simulates exactly one or several trajectories of a Meixner process, homogeneous or not, at given inspection times.

Usage

```
sim_meixner(t, a, b, d, m, Lambda = function(t) t, n_traj = 1)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector, for instance `seq(0, 10, by = 0.5)`: all the trajectories share these times, and their number is `n_traj`; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` is then ignored); • the data frame returned by `inspection_times()`, which gives one plan per trajectory.
`a`	Scale parameter, $a > 0$.
`b`	Asymmetry parameter, $-\pi < b < \pi$: $b = 0$ gives symmetric increments, $b > 0$ increments skewed to the right.
`d`	Shape parameter per unit of Λ, $d > 0$.
`m`	Location parameter per unit of Λ, real.
`Lambda`	Function of time: an R function of `t`, continuous and strictly increasing with $\Lambda(0) = 0$, for instance `function(t) t^1.5`. The default `function(t) t` gives the homogeneous process.
`n_traj`	Number of trajectories to simulate when `t` is a vector (default 1).

Value

An object of class `levysim`, whose form depends on the inspection times:

- **same times for all the trajectories** (`t` a vector, or a plan of `inspection_times()` with the same times for all the trajectories): a matrix with columns `t` (inspection times), `X1`, ..., `Xn` (simulated degradation levels of the trajectories 1 to n, equal to 0 at `t = 0`). The k-th trajectory is `x[, k + 1]` or `x[, "Xk"]`;
- **different times** (`t` a list, or a plan of `inspection_times()` with different times): a list with one data frame per trajectory, with columns `t` and `X`. The k-th trajectory is `x[[k]]`.

Printing the object shows the model, the parameters and the first rows. Use `traj_df()` to gather the data in a single data frame (with the increments if needed) and `plot_traj()` to plot the trajectories.

Model

The $Meixner(a, b, d, m)$ distribution, $a > 0$, $-\pi < b < \pi$, $d > 0$, m real, has density

$$f(x) = \frac{(2\cos(b/2))^{2d}}{2a\pi\,\Gamma(2d)} \exp\left(\frac{b(x-m)}{a}\right)\left|\Gamma\left(d + i\frac{x-m}{a}\right)\right|^2.$$

Let Λ be continuous, strictly increasing, with $\Lambda(0) = 0$. The Meixner process has $X(0) = 0$, independent increments and

$$X(t) - X(s) \sim Meixner(a, b, d\Delta\Lambda, m\Delta\Lambda).$$

Hence $E[X(t)] = (m + ad\tan(b/2))\Lambda(t)$ and $Var(X(t)) = a^2 d\Lambda(t)/(2\cos^2(b/2))$. The trajectories are not monotone, purely made of jumps, of infinite variation; b sets the asymmetry and a is a scale parameter. With $\Lambda(t) = t$ (default), it is the homogeneous Meixner process, a Lévy process.

Exact simulation

There is no Meixner generator in `stats`. Each increment, of law $Meixner(a, b, d\Delta\Lambda_i, m\Delta\Lambda_i)$, is drawn by numerical inversion of its distribution function (method PINV of the package `Runuran`, from the exact density, error in u of order 10^{-10}), then the increments are cumulated. One generator is built for each distinct length $\Delta\Lambda_i$ and reused for all the trajectories.

References

Schoutens, W. (2003). *Lévy Processes in Finance: Pricing Financial Derivatives*. Wiley.

Schoutens, W. and Teugels, J. L. (1998). Lévy processes, polynomials and martingales. *Communications in Statistics. Stochastic Models*, 14(1-2), 335-349.

See Also

`plot_traj()`, `traj_df()`, `inspection_times()`, `plot_plan()`

Examples

```
# One trajectory inspected every 0.5 time unit up to 10
t <- seq(0, 10, by = 0.5)
set.seed(1)
x <- sim_meixner(t, a = 1, b = 0.3, d = 1.5, m = 0.2)
x
plot_traj(x)

# Ten trajectories with the same inspection times
x <- sim_meixner(t, a = 1, b = 0.3, d = 1.5, m = 0.2, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])
x[, "X2"]

# Irregular inspections, frequent in the middle of the period
plan <- inspection_times("inc-dec", n = 20, n_traj = 5, tmax = 10)
x <- sim_meixner(plan, a = 1, b = 0.3, d = 1.5, m = 0.2)
plot_traj(x)
```

```
# Different inspection times for each trajectory: a list of data frames
x <- sim_meixner(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), a = 1, b = 0.3, d = 1.5, m = 0.2)
x[[2]]

# Non-homogeneous process with Lambda(t) = t^1.5
x <- sim_meixner(t, a = 1, b = 0.3, d = 1.5, m = 0.2, Lambda = function(t) t^1.5, n_traj = 10)
plot_traj(x, "Non-homogeneous Meixner process")

# All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))
```

`sim_vg` *Simulation of the variance-gamma process*

Description

Simulates exactly one or several trajectories of a variance-gamma (VG) process, homogeneous or not, at given inspection times, by subordination.

Usage

```
sim_vg(
  t,
  theta,
  sigma,
  nu,
  Lambda = function(t) t,
  method = "subordination",
  n_traj = 1
)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector, for instance `seq(0, 10, by = 0.5)`: all the trajectories share these times, and their number is `n_traj`; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` is then ignored); • the data frame returned by `inspection_times()`, which gives one plan per trajectory.
`theta`	Drift of the subordinated Brownian motion, real: mean degradation per unit of Λ.
`sigma`	Diffusion coefficient of the subordinated Brownian motion, $\sigma > 0$ (standard deviation, not variance).
`nu`	Variance rate of the gamma clock, $\nu > 0$: the larger ν, the larger the jumps; as $\nu \to 0$ the process becomes a Wiener process.
`Lambda`	Function of time: an R function of `t`, continuous and strictly increasing with $\Lambda(0) = 0$, for instance `function(t) t^1.5`. The default `function(t) t` gives the homogeneous process.

`method`	Simulation algorithm: `"subordination"` (default) or `"difference"` (difference of two gamma processes). Both are exact and give the same distribution; see the sections above.
`n_traj`	Number of trajectories to simulate when `t` is a vector (default 1).

Value

An object of class `levysim`, whose form depends on the inspection times:

- **same times for all the trajectories** (`t` a vector, or a plan of `inspection_times()` with the same times for all the trajectories): a matrix with columns `t` (inspection times), `X1`, ..., `Xn` (simulated degradation levels of the trajectories 1 to n, equal to 0 at `t = 0`). The k-th trajectory is `x[, k + 1]` or `x[, "Xk"]`;
- **different times** (`t` a list, or a plan of `inspection_times()` with different times): a list with one data frame per trajectory, with columns `t` and `X`. The k-th trajectory is `x[[k]]`.

Printing the object shows the model, the parameters and the first rows. Use `traj_df()` to gather the data in a single data frame (with the increments if needed) and `plot_traj()` to plot the trajectories.

Model

Let Λ be continuous, strictly increasing, with $\Lambda(0) = 0$, θ real, $\sigma > 0$ and $\nu > 0$. Let G be a gamma process with $G(t) - G(s) \sim \Gamma(\Delta\Lambda/\nu, \text{rate } 1/\nu)$, so that $E[G(t)] = \Lambda(t)$ and $Var(G(t)) = \nu\Lambda(t)$, and B an independent standard Brownian motion. The VG process is the Brownian motion with drift evaluated at the random clock G:

$$X(t) = \theta G(t) + \sigma B(G(t)).$$

It has independent increments, $E[X(t)] = \theta\Lambda(t)$ and $Var(X(t)) = (\sigma^2 + \theta^2\nu)\Lambda(t)$. The trajectories are not monotone, purely made of jumps, of finite variation. With $\Lambda(t) = t$ (default), it is the homogeneous VG process, a Lévy process. As $\nu \to 0$, the VG process tends to the Wiener process $\theta\Lambda(t) + \sigma B(\Lambda(t))$.

Exact simulation 1

subordination (`method = "subordination"`): Conditionally on the clock increment ΔG, $X(t) - X(s) \sim N(\theta\Delta G, \sigma^2\Delta G)$:

1. Draw independently $\Delta G_i \sim \Gamma(\Delta\Lambda_i/\nu)$ with scale ν.
2. $\Delta X_i = \theta\Delta G_i + \sigma\sqrt{\Delta G_i}\, Z_i$, with Z_i independent $N(0, 1)$.
3. Cumulative sum.

This method also gives the random clock G, which is useful to interpret the process.

Exact simulation 2

difference of two gamma processes (`method = "difference"`): Let

$$\mu_\pm = \frac{1}{2}\sqrt{\theta^2 + \frac{2\sigma^2}{\nu}} \pm \frac{\theta}{2} > 0,$$

and G_+, G_- two independent gamma processes with shape function $\Lambda(t)/\nu$ and rates $1/(\nu\mu_\pm)$. Then X has the same distribution as $G_+ - G_-$: the characteristic function of $\Delta G_+ - \Delta G_-$ is

$$(1 - iu\nu\mu_+)^{-\Delta\Lambda/\nu}(1 + iu\nu\mu_-)^{-\Delta\Lambda/\nu} = (1 - iu\nu(\mu_+ - \mu_-) + u^2\nu^2\mu_+\mu_-)^{-\Delta\Lambda/\nu},$$

with $\mu_+ - \mu_- = \theta$ and $\nu^2\mu_+\mu_- = \sigma^2\nu/2$, which is the characteristic function $(1 - iu\theta\nu + \sigma^2\nu u^2/2)^{-\Delta\Lambda/\nu}$ of the VG increment.

1. Draw independently $\Delta G_i^+ \sim \Gamma(\Delta\Lambda_i/\nu)$ with scale $\nu\mu_+$ and $\Delta G_i^- \sim \Gamma(\Delta\Lambda_i/\nu)$ with scale $\nu\mu_-$.
2. $\Delta X_i = \Delta G_i^+ - \Delta G_i^-$.
3. Cumulative sum.

This method only uses gamma draws. Both methods are exact and give the same distribution.

References

Madan, D. B., Carr, P. P. and Chang, E. C. (1998). The variance gamma process and option pricing. *European Finance Review*, 2(1), 79-105. doi:10.1023/A:1009703431535

See Also

plot_traj(), traj_df(), inspection_times(), plot_plan()

Examples

```
# One trajectory inspected every 0.5 time unit up to 10
t <- seq(0, 10, by = 0.5)
set.seed(1)
x <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8)
x
plot_traj(x)

# Ten trajectories, simulated by subordination (default)
x <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])
x[, "X2"]

# The same model simulated as a difference of two gamma processes
y <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8, method = "difference", n_traj = 10)
plot_traj(y, "Variance-gamma process, difference of two gamma processes")

# Irregular inspections, more and more frequent over time
plan <- inspection_times("increasing", n = 20, n_traj = 5, tmax = 10)
x <- sim_vg(plan, theta = 0.5, sigma = 1.2, nu = 0.8)
plot_traj(x)

# Different inspection times for each trajectory: a list of data frames
x <- sim_vg(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), theta = 0.5, sigma = 1.2, nu = 0.8)
x[[2]]

# Non-homogeneous process with Lambda(t) = t^1.5
x <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8, Lambda = function(t) t^1.5, n_traj = 10)
plot_traj(x, "Non-homogeneous variance-gamma process")

# All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))
```

`sim_wiener` *Simulation of the Wiener process*

Description

Simulates exactly one or several trajectories of a Wiener process with drift, homogeneous or with time transformations, at given inspection times.

Usage

```
sim_wiener(
  t,
  mu,
  sigma,
  Lambda = function(t) t,
  tau = function(t) t,
  n_traj = 1
)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector, for instance `seq(0, 10, by = 0.5)`: all the trajectories share these times, and their number is `n_traj`; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` is then ignored); • the data frame returned by `inspection_times()`, which gives one plan per trajectory.
`mu`	Drift, real: mean degradation per unit of Λ.
`sigma`	Diffusion coefficient, $\sigma > 0$: standard deviation (not variance) per square root of unit of τ.
`Lambda`	Function of time for the drift: an R function of `t`, continuous with $\Lambda(0) = 0$, for instance `function(t) t^1.5`. The default `function(t) t` gives a linear mean.
`tau`	Function of time for the diffusion: an R function of `t`, strictly increasing with $\tau(0) = 0$. The default `function(t) t` gives the usual Brownian motion.
`n_traj`	Number of trajectories to simulate when `t` is a vector (default 1).

Value

An object of class `levysim`, whose form depends on the inspection times:

- **same times for all the trajectories** (`t` a vector, or a plan of `inspection_times()` with the same times for all the trajectories): a matrix with columns `t` (inspection times), `X1`, ..., `Xn` (simulated degradation levels of the trajectories 1 to n, equal to 0 at `t = 0`). The k-th trajectory is `x[, k + 1]` or `x[, "Xk"]`;
- **different times** (`t` a list, or a plan of `inspection_times()` with different times): a list with one data frame per trajectory, with columns `t` and `X`. The k-th trajectory is `x[[k]]`.

Printing the object shows the model, the parameters and the first rows. Use `traj_df()` to gather the data in a single data frame (with the increments if needed) and `plot_traj()` to plot the trajectories.

Model

Let μ be real, $\sigma > 0$, Λ a continuous function with $\Lambda(0) = 0$, τ a continuous strictly increasing function with $\tau(0) = 0$, and B a standard Brownian motion. The process is

$$X(t) = \mu\Lambda(t) + \sigma B(\tau(t)),$$

with independent increments $X(t) - X(s) \sim N(\mu\Delta\Lambda, \sigma^2\Delta\tau)$. Hence $E[X(t)] = \mu\Lambda(t)$ and $Var(X(t)) = \sigma^2\tau(t)$. The trajectories are continuous and not monotone. With $\Lambda(t) = \tau(t) = t$ (default), it is the homogeneous Wiener process $\mu t + \sigma B(t)$, a Lévy process; $\Lambda = \tau$ gives the usual Wiener process with transformed time.

Exact simulation

$\Delta X_i = \mu\Delta\Lambda_i + \sigma\sqrt{\Delta\tau_i}\, Z_i$, with Z_i independent $N(0,1)$, then cumulative sum.

References

Whitmore, G. A. (1995). Estimating degradation by a Wiener diffusion process subject to measurement error. *Lifetime Data Analysis*, 1, 307-319.

See Also

plot_traj(), traj_df(), inspection_times(), plot_plan()

Examples

```
# One trajectory inspected every 0.5 time unit up to 10
t <- seq(0, 10, by = 0.5)
set.seed(1)
x <- sim_wiener(t, mu = 0.5, sigma = 1.5)
x
plot_traj(x)

# Ten trajectories with the same inspection times
x <- sim_wiener(t, mu = 0.5, sigma = 1.5, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])
x[, "X2"]

# Irregular inspections, frequent at the start and at the end
plan <- inspection_times("dec-inc", n = 20, n_traj = 5, tmax = 10)
x <- sim_wiener(plan, mu = 0.5, sigma = 1.5)
plot_traj(x)

# Different inspection times for each trajectory: a list of data frames
x <- sim_wiener(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), mu = 0.5, sigma = 1.5)
x[[2]]

# Transformed time: Lambda(t) = t^1.5 and tau(t) = t^1.2
x <- sim_wiener(t, mu = 0.5, sigma = 1.5, Lambda = function(t) t^1.5,
                tau = function(t) t^1.2, n_traj = 10)
plot_traj(x, "Wiener process with transformed time")
```

```
# All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))
```

`traj_df` *Simulated data as a data frame*

Description

Gathers the trajectories simulated by a `sim_` function into a single data frame, one row per inspection, ready to be saved, summarized or used by other functions. Optionally adds the time steps and the increments between two consecutive inspections.

Usage

```
traj_df(x, increments = FALSE)
```

Arguments

`x`	Result of a simulation function of the package, for instance `sim_gamma()`.
`increments`	`FALSE` (default) for the levels only; `TRUE` to add the columns `dt` and `dX`.

Value

A data frame with one row per inspection and the columns:

- `id`: number of the trajectory (1, 2, ...);
- `t`: inspection time;
- `X`: simulated degradation level at time `t`;
- with `increments = TRUE`, also `dt`, the time elapsed since the previous inspection, and `dX`, the increment of the level since the previous inspection (`NA` at the first inspection, `t = 0`, of each trajectory).

To work on a single trajectory, use `subset(traj_df(x), id == k)`.

See Also

`plot_traj()`

Examples

```
set.seed(1)
x <- sim_gamma(seq(0, 10, by = 1), a = 2, b = 3, n_traj = 3)
d <- traj_df(x)
head(d)

# Degradation level of each trajectory at the last inspection
d[d$t == 10, ]

# Time steps and increments between two inspections
head(traj_df(x, increments = TRUE))
```

```
# Trajectories with different inspection times
y <- sim_gamma(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8)), a = 2, b = 3)
traj_df(y, increments = TRUE)

# Save the simulated data in a csv file
f <- tempfile(fileext = ".csv")
write.csv(d, f, row.names = FALSE)
```

Index

∗ **package**
levysim-package, 2

inspection_times, 4
inspection_times(), *2, 3*, *7–9*, *11–21*

levysim *(*levysim-package*)*, 2
levysim-package, 2

plot_plan, 6
plot_plan(), *2*, *5*, *9*, *12*, *14*, *16*, *19*, *21*
plot_traj, 7
plot_traj(), *2*, *7*, *9*, *11–16*, *18–22*

sim_gamma, 8
sim_gamma(), *2, 3*, *8*, *22*
sim_ig, 10
sim_ig(), *2*
sim_igam, 12
sim_igam(), *2*
sim_meixner, 15
sim_meixner(), *2*
sim_vg, 17
sim_vg(), *2*
sim_wiener, 20
sim_wiener(), *2*
stats::rgamma(), *2*
stats::rnorm(), *2*

traj_df, 22
traj_df(), *2*, *8, 9*, *11–16*, *18–21*